DATA STRUCTURES & ALGORITHMS

MINI PROJECT

University Student Management System

A C++ console-based academic project focused on dynamic student records, searching, Bubble Sort, and clean input/output.

C++ DYNAMIC ARRAYS SEARCH & SORT

PREPARED BY / GROUP 01

01	Muhammad Salman	SAP ID 72373
02	Huzaifa Altaf Abbasi	SAP ID —
03	Muhammad Talha	SAP ID —

SUBMITTED TO Mueed Ahmad

01 / OVERVIEW

Project Introduction

The actual implementation and presentation both describe a menu-driven student record management program built with C++.

PROJECT PURPOSE

Manage student information in a dynamically allocated array, with options to add, display, find and organize records through a console menu.

IMPLEMENTED CAPABILITIES

01 Add records

Store student ID, full name, semester and CGPA.

02 Display records

Present saved students in aligned console columns.

03 Find by ID

Locate a student using the entered student ID.

04 Sort by ID

Reorder records in ascending student ID order.

05 Sort by CGPA

Reorder records from highest to lowest CGPA.

06 Array status

Display current count, capacity and free slots.

02 / ARCHITECTURE

Data Structure & Storage

The code combines one Student structure with a dynamically allocated array and two tracking variables.

Student record model

ACTUAL STRUCT DEFINITION

```
struct Student
{
    int id;
    string name;
    int semester;
    float cgpa;
};
```

FIELDS

id	integer
name	string
semester	integer
cgpa	float

Dynamic tracking variables

```
Student* students = new Student[2];

studentCount = 0                    capacity = 2
```

How storage expands

INITIAL CAPACITY



AFTER ADDING 3RD RECORD



When studentCount reaches capacity, the program allocates a new array with twice the space, copies existing records, releases the old array, and updates the pointer.

03 / USER WORKFLOW

Console Menu & Functions

The actual main() function presents six operations and an exit option inside a repeated menu loop.

MENU OPTION	FUNCTION IN SOURCE
01	Add Student — Requests the four Student fields and checks for duplicate IDs.	`addStudent()`
02	Display All Students — Prints the stored records in a formatted table.	`displayStudents()`
03	Search Student by ID — Uses searchStudent(id) to find a matching record.	`findStudent()`
04	Sort Students by ID — Bubble Sort with IDs in ascending order.	`sortByID()`
05	Sort Students by CGPA — Bubble Sort with CGPA in descending order.	`sortByCGPA()`
06	Show Array Count / Capacity — Prints count, capacity and remaining space.	`showArrayInfo()`

0 / Exit

The program prints "Program closed." and releases dynamically allocated memory before returning from main().

04 / PROBLEM ONE

Dynamic Array Expansion

The presentation highlights a capacity issue when a third student is added to an array initially holding two records.

Problem & solution

PROBLEM

An initial capacity of 2 is not enough for a third record. The program must grow the array without losing existing student information.

Implementation in the C++ file

C++ SOURCE EXCERPT

```
void increaseCapacity()
{
    int newCapacity = capacity * 2;
    Student* newArray = new Student[newCapacity];
    for (int i = 0; i < studentCount; i++)
    {
        newArray[i] = students[i];
    }
    delete[] students;
    students = newArray;
    capacity = newCapacity;
}
```

PROCESS

01	**Double**	Calculate newCapacity = capacity * 2.
02	**Transfer**	Copy the current Student objects into newArray.
03	**Replace**	Delete the old allocation and update students and capacity.

AI PROMPT REFERENCED IN THE PRESENTATION

"Explain step by step how I can increase array capacity, copy records into a new array, delete old memory, and update pointers."

05 / PROBLEM TWO

Resolving getline() Skips

The presentation documents an input issue that occurs after reading a numeric student ID with cin.

THE ISSUE

After cin reads an integer, a leftover newline may remain in the input stream. A subsequent getline() can consume that newline instead of waiting for the full student name.

Exact input-handling code

C++ SOURCE EXCERPT

```
cin >> s.id;

cin.ignore(numeric_limits<streamsize>::max(), '\n');

cout << "Enter Student Name: ";
getline(cin, s.name);
```

Why it works

1. **Read the ID**
   cin >> s.id reads the numeric student ID.

2. **Clear the remaining line**
   cin.ignore(...) discards leftover characters up to the newline.

3. **Read the complete name**
   getline(cin, s.name) accepts spaces in the entered student name.

AI PROMPT REFERENCED IN THE PRESENTATION

"Explain why getline skips after cin and how #include and cin.ignore(...) fix the input buffer problem."

```
Source header:  #include <limits>
```

06 / PROBLEM THREE

Formatted Console Output

The presentation focuses on aligning student record columns and displaying CGPA consistently with two decimal places.

Printing a clean student table

C++ SOURCE EXCERPT

```
cout << left
    << setw(12) << "ID"
    << setw(22) << "Name"
    << setw(12) << "Semester"
    << setw(10) << "CGPA" << endl;

for (int i = 0; i < studentCount; i++)
{
    cout << left
        << setw(12) << students[i].id
        << setw(22) << students[i].name
        << setw(12) << students[i].semester
        << fixed << setprecision(2)
        << students[i].cgpa << endl;
}
```

OUTPUT MANIPULATORS

`left`

Aligns printed values to the left.

`setw(...)`

Reserves a defined width for each column.

`fixed`

Uses fixed-point decimal formatting.

`setprecision(2)`

Displays CGPA with two decimal places.

AI PROMPT REFERENCED IN THE PRESENTATION

"Explain why I should use #include and how setw(), left, fixed, and setprecision(2) work."

Source header: `#include <iomanip>`

07 / ALGORITHMS

Searching & Bubble Sort

Student search and both sorting options operate on the same dynamically allocated Student array.

Searching by ID

```
searchStudent(int id)
```

The function checks student IDs sequentially, returns the matching index, and returns -1 when no matching record exists. findStudent() uses that result for console feedback.

Bubble Sort: two implemented orders

`sortByID()`

ID: ASCENDING

```
students[j].id >
students[j + 1].id
```

`sortByCGPA()`

CGPA: HIGHEST FIRST

```
students[j].cgpa <
students[j + 1].cgpa
```

Both functions compare adjacent records and exchange complete Student objects using a temporary Student variable.

VERIFIED SAMPLE: CGPA SORT

ID	NAME	CGPA
103	Muhammad Talha	3.89
101	Muhammad Salman	3.76
102	Huzaifa Altaf Abbasi	3.62

Illustrative inputs run against the supplied C++ program.

08 / RESULTS

Execution & Conclusion

The C++ file was compiled and exercised with three sample records to check the reported functionality.

OBSERVED ARRAY STATUS AFTER THREE ADDITIONS

3	4	1
Student records	Array capacity	Available space

Example formatted output

ID	Name	Semester	CGPA
101	Muhammad Salman	3	3.76
102	Huzaifa Altaf Abbasi	3	3.62
103	Muhammad Talha	3	3.89

Conclusion & key takeaways

1. **Dynamic memory management**
   The array starts at capacity 2, grows by doubling, and releases old allocations.

2. **Searching and sorting**
   The program searches by ID and sorts students using Bubble Sort by ID or CGPA.

3. **Input and display**
   The code handles full-name input and prints neatly aligned records with two-decimal CGPA.

REPORT CONTENT BASED ON: PROVIDED C++ SOURCE + PRESENTATION